Semantic Chunking

Using Vector Mathematics to Detect Topic Shifts

Section 2 Upgrade · RAG From Scratch

The first iteration of this RAG system used **fixed-size chunking** — cutting text every n words regardless of meaning. The upgraded `_chunk_semantic` method replaces this with a mathematically principled approach: it uses **cosine similarity between sentence vectors** to detect when a topic changes, and only then creates a new chunk boundary.

The Problem with Fixed-Size Chunking

The Problem with Fixed-Size Chunking

Consider a document containing these three sentences:

ID	Sentence	Topic
S_1	*"Apples are a tasty and sweet fruit."*	Fruit
S_2	*"Bananas are also a great sweet fruit."*	Fruit
S_3	*"My car needs an oil change today."*	Vehicles

A fixed-word chunker might bundle the tail of the fruit discussion with the car sentence into the same chunk — poisoning the retrieved context with irrelevant information. Semantic chunking uses vector math to **guarantee** that S_1 and S_2 stay together, and S_3 is isolated.

Step 1 — Vectorise the Sentences

The text is first split into sentences using regular expressions. Each sentence is then passed through an `EmbeddingLayer` that maps it to a dense vector in $\mathbb{R}^n$. For clarity, we work in $\mathbb{R}^3$ and assume the vectors are **unit-normalised** (magnitude = 1):

Sentence Vectors (3-dimensional, normalised)

$$\vec{v}_1 = [\ 0.8,\ 0.6,\ 0.0\ ] \qquad (S_1\text{: Apples...})$$
$$\vec{v}_2 = [\ 0.7,\ 0.7,\ 0.1\ ] \qquad (S_2\text{: Bananas...})$$
$$\vec{v}_3 = [\ 0.0,\ 0.1,\ 0.9\ ] \qquad (S_3\text{: Car...})$$

Notice how $\vec{v}_1$ and $\vec{v}_2$ have large components in the first two dimensions (fruit-related concepts), while $\vec{v}_3$ has nearly all its weight in the third dimension (vehicle-related concepts).

Step 2 — Calculate Sequential Cosine Similarity

The code loops through all consecutive sentence pairs and computes the **cosine similarity** between each adjacent pair $(\vec{v}_i,\ \vec{v}_{i+1})$:

$$\cos(\vec{v}_i,\ \vec{v}_{i+1}) = \frac{\vec{v}_i \cdot \vec{v}_{i+1}}{\|\vec{v}_i\|_2 \times \|\vec{v}_{i+1}\|_2}$$

Because all vectors are unit-normalised ($\|\vec{v}\|_2 = 1$), the formula reduces to the bare dot product:

$$\cos(\vec{v}_i,\ \vec{v}_{i+1}) = \vec{v}_i \cdot \vec{v}_{i+1} = \sum_k v_{i,k} \cdot v_{i+1,k}$$

Pair 1 — S_1 (Apples) vs. S_2 (Bananas)

$\cos(\vec{v}_1, \vec{v}_2)$

$$\vec{v}_1 \cdot \vec{v}_2 = (0.8 \times 0.7) + (0.6 \times 0.7) + (0.0 \times 0.1) = 0.56 + 0.42 + 0.00$$

$$\boxed{\cos(\vec{v}_1, \vec{v}_2) = \mathbf{0.98}}$$

A score of 0.98 is extremely high — the two sentence vectors point in almost the same direction. **Same topic.**

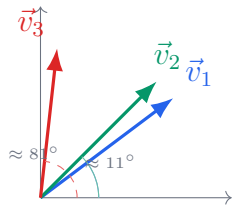
Pair 2 — S_2 (Bananas) vs. S_3 (Car)

$\cos(\vec{v}_2, \vec{v}_3)$

$$\vec{v}_2 \cdot \vec{v}_3 = (0.7 \times 0.0) + (0.7 \times 0.1) + (0.1 \times 0.9) = 0.00 + 0.07 + 0.09$$

$$\boxed{\cos(\vec{v}_2, \vec{v}_3) = \mathbf{0.16}}$$

A score of 0.16 is very low — the vectors are nearly *orthogonal*. **Topic has changed.**



Small angle $\Rightarrow$ high similarity

$\vec{v}_1$ and $\vec{v}_2$ are nearly aligned ($\approx 11°$ apart, similarity 0.98).
$\vec{v}_2$ and $\vec{v}_3$ are nearly orthogonal ($\approx 81°$ apart, similarity 0.16).

Step 3 — The Drop-Off Threshold

The `_chunk_semantic` function accepts a **threshold** parameter τ (default $\tau = 0.5$). The decision rule is:

$$\cos(\vec{v}_i,\, \vec{v}_{i+1}) \geq \tau \quad \Longrightarrow \quad \text{Sentences belong to the } \textbf{same chunk}$$
$$\cos(\vec{v}_i,\, \vec{v}_{i+1}) < \tau \quad \Longrightarrow \quad \textbf{Split}\text{: close the current chunk, open a new one}$$

Applying $\tau = 0.5$ to our two similarity scores:

Pair	Similarity	Threshold	Pass?	Action
$S_1 \to S_2$	0.98	≥ 0.5	**Yes**	Keep S_2 in the current chunk
$S_2 \to S_3$	0.16	< 0.5	**No**	**Split!** Start a new chunk at S_3

Final Result

Chunk	Content
Chunk 1	*"Apples are a tasty and sweet fruit. Bananas are also a great sweet fruit."* **[Fruit topic]**
Chunk 2	*"My car needs an oil change today."* **[Vehicle topic]**

The system has **mathematically guaranteed** that no chunk mixes the fruit discussion with the car sentence. The chunk boundary was placed *exactly* where the cosine similarity dropped below the threshold — at the transition between concepts.

Why This Is Better: A Comparison

Property	Fixed-Size Chunking	Semantic Chunking
Boundary criterion	Word count	Topic similarity
Requires embeddings	No	Yes
Preserves meaning	Sometimes	Always
Mixed-topic chunks	Possible	Prevented
Tunable parameter	Chunk size (c), overlap (o)	Threshold (τ)

The key insight is that the **dot product** of two normalised vectors is a direct, computationally cheap measure of their semantic alignment — making it ideal for a tight inner loop over sentence pairs.

Choosing the right threshold τ is an empirical decision. A lower τ produces fewer, larger chunks (tolerating more topic drift); a higher τ produces many fine-grained chunks (splitting at subtle shifts). In practice, $\tau \in [0.4, 0.7]$ works well for general-purpose retrieval.